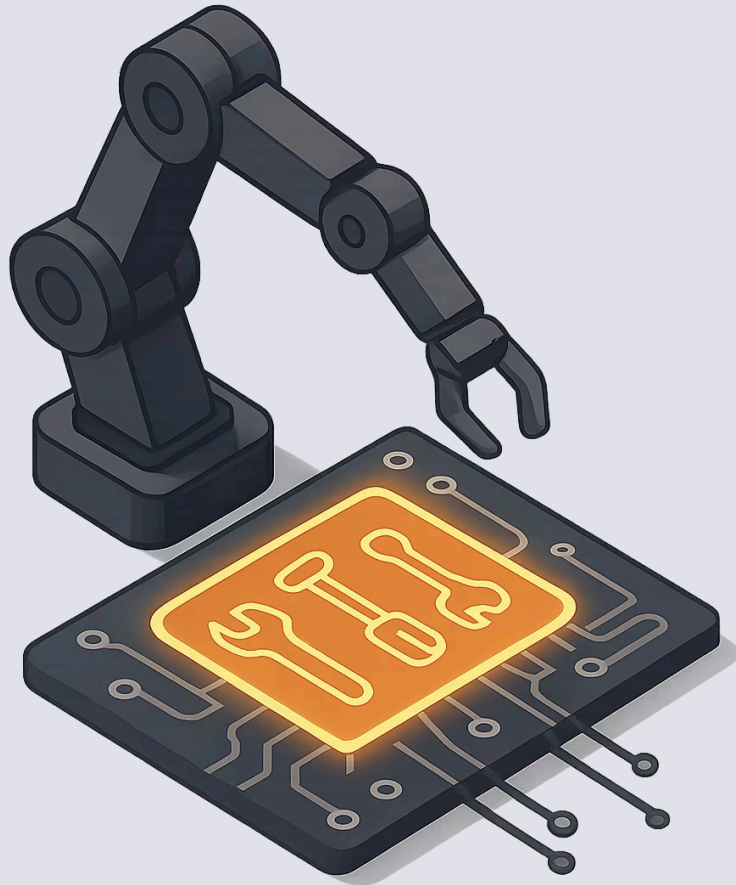




Development of a Tool-Using AI Agent

From Traditional Programs to Intelligent Agentic AI

Dr. Anilkumar V. Brahmane

Computer Engineering | Sanjivani College of Engineering

TY B.TECH ASSIGNMENT GUIDE

Assignment Overview

What Are We Building?

1 Understand the Question

The agent reads and interprets what the user is asking.

2 Decide & Select a Tool

It decides whether a Calculator or Search API is needed.

3 Execute & Respond

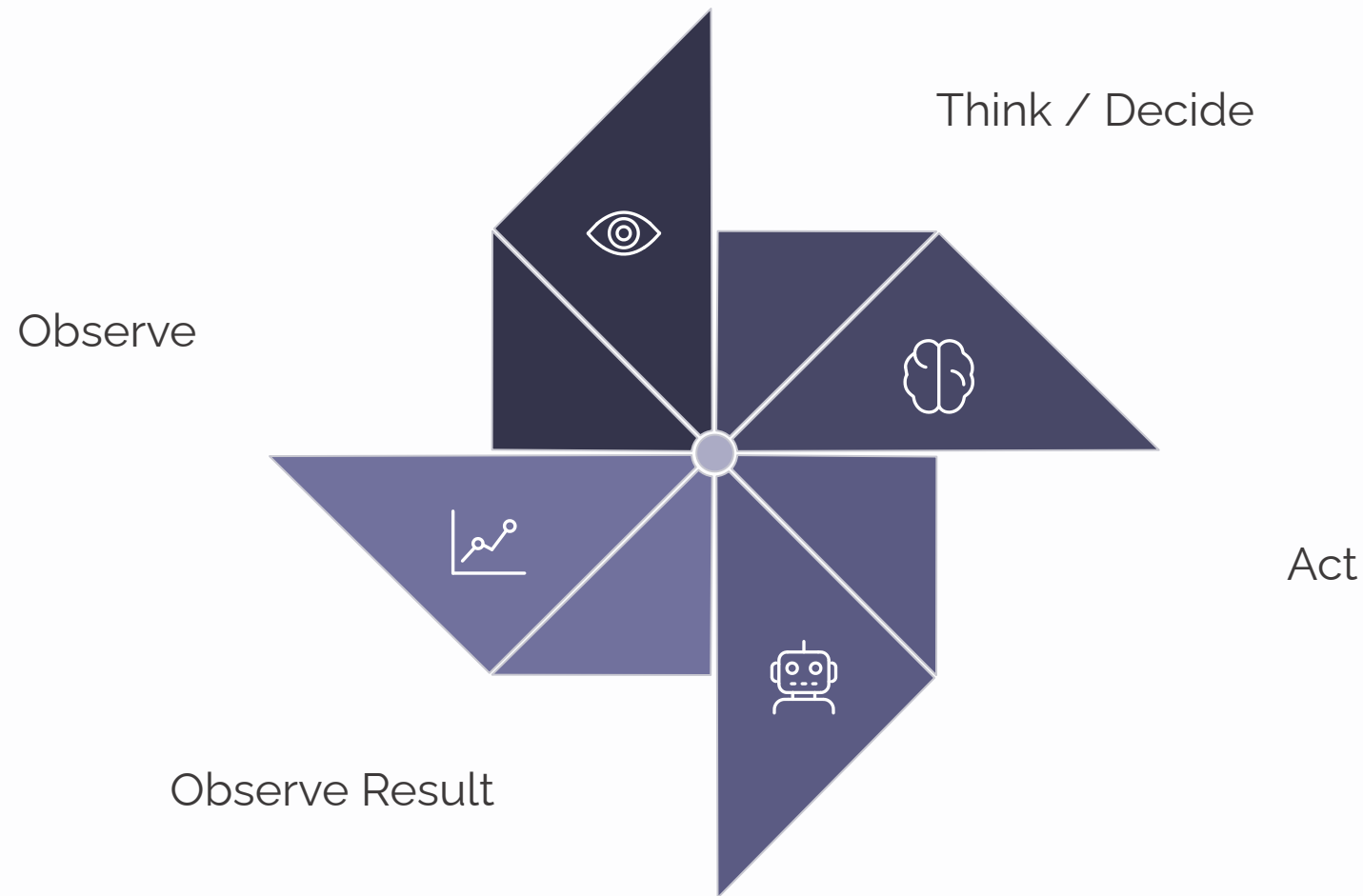
It uses the tool, collects results, and gives a final answer.

📌 This is more than asking ChatGPT a question — it is a system that **thinks, decides, and acts**.

Core Concept

What Is an AI Agent?

An AI agent is a software system that follows a continuous loop of perception and action:



Example: Asked "Calculate total cost of 15 notebooks at ₹80 each," the agent does not just multiply – it **understands the task, selects the Calculator tool, gets the result (₹1200), and then generates a natural-language answer.**

What Makes It an "Agent"?

Simple Program

Always performs the same fixed operation. No decision-making.
Example: always multiplies two numbers.

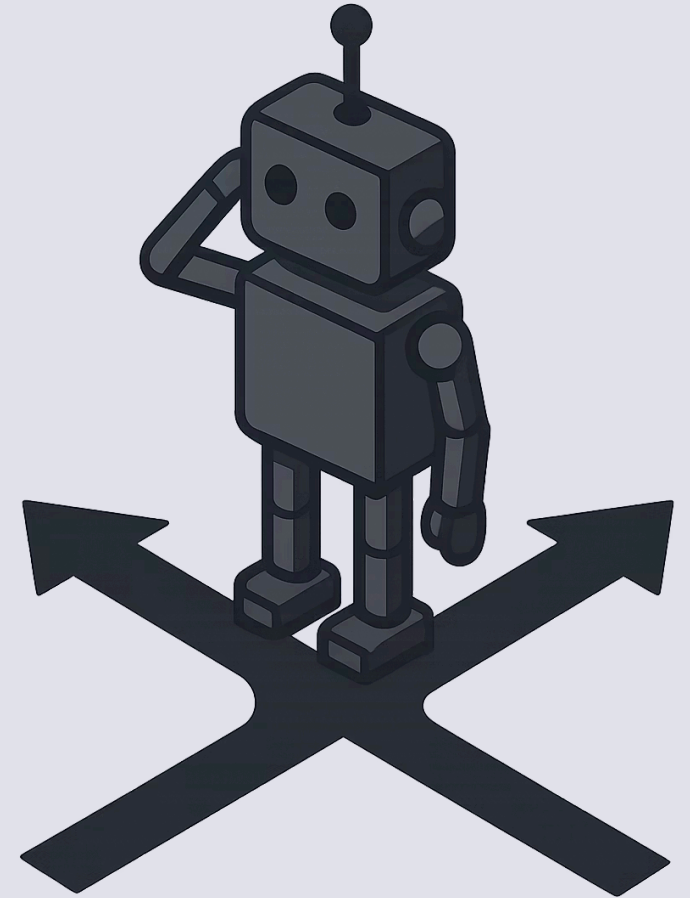
```
a = 15  
b = 80  
print(a * b)
```

AI Agent

Understands different requests and **decides** what action is appropriate:

- $25 \times 40 \rightarrow$ Use **Calculator**
- Capital of Japan? $\rightarrow$ Use **Search Tool**
- 15% of ₹800 $\rightarrow$ Use **Calculator**

Tool selection is the key characteristic.



Building Blocks

What Is a Tool?

A **tool** is an external function or service the AI agent calls to complete a specific task. For this assignment, start with two tools:



Calculator Tool

- Addition, Subtraction
- Multiplication, Division
- Percentage & Average



Search API Tool

- Finding current information
- Definitions & Facts
- Real-world data retrieval

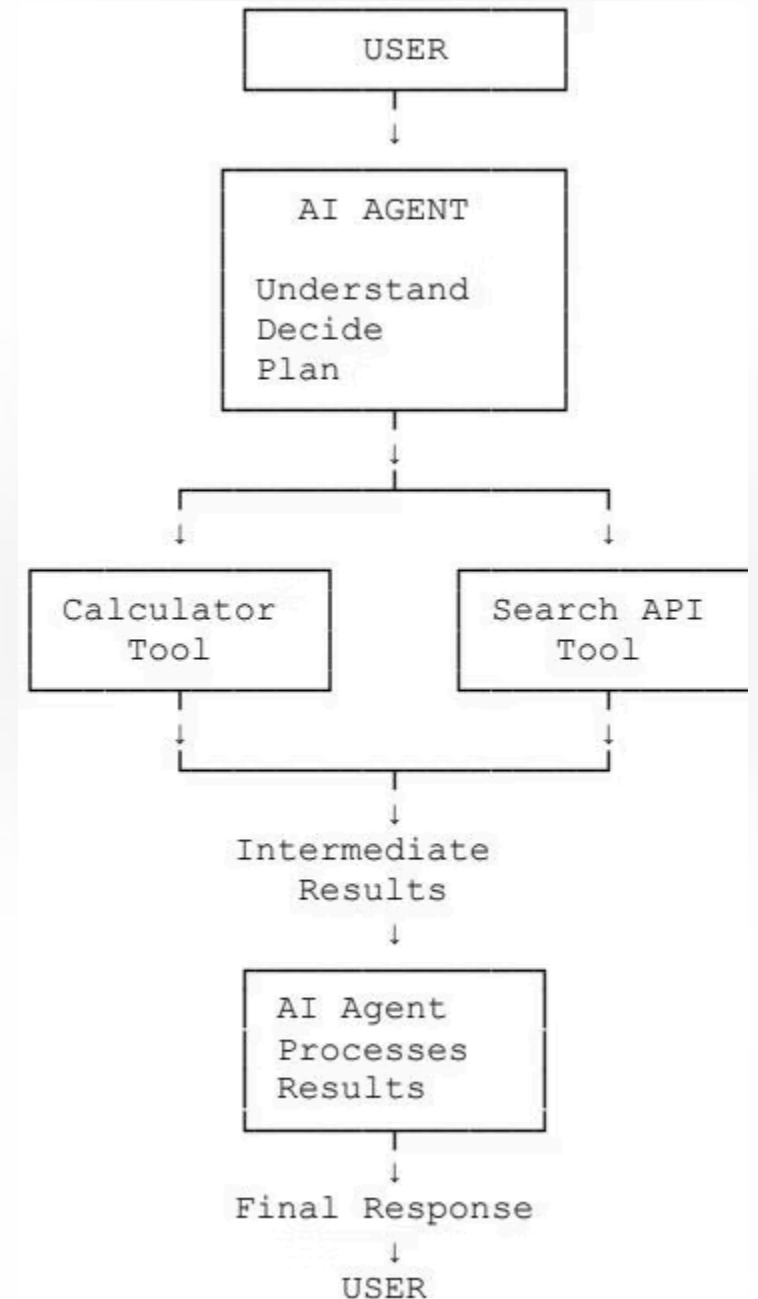
Other tools possible: Weather API, Database, Python Interpreter, Email API, Maps API.

System Design

Basic Architecture of the Agent

The user sends a question → the **AI Agent understands, decides, and plans** → routes to Calculator or Search API → collects **intermediate results** → processes them → delivers the **Final Response** back to the user.

— Dr. Anilkumar V. Brahmane | Computer Engineering | Sanjivani College of Engineering



Key Concept

Single-Step vs. Multi-Step Problems

Single-Step

One action solves the problem.

Example: Calculate 25×10

Question → Calculator → Answer ✓

Multi-Step

Example: *"Population of India – calculate after 10% increase."*

- **Step 1:** Search API → Population = X
- **Step 2:** Calculator → $X \times 1.10$ = New Population
- **Step 3:** Combine → Final Response

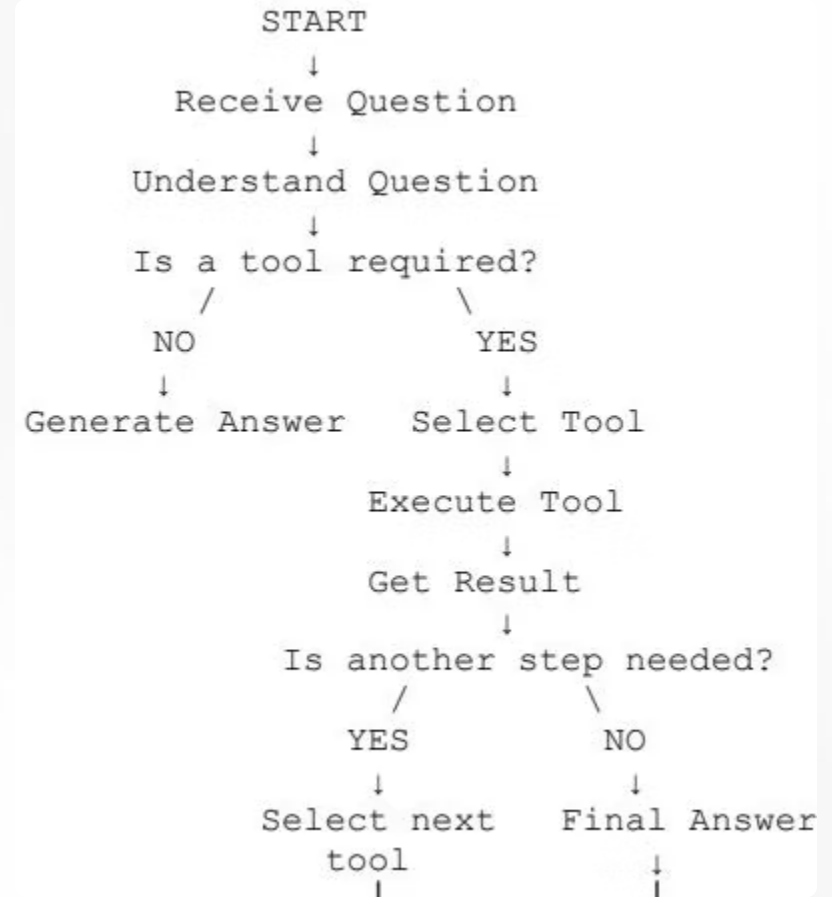
This is why it is called a **multi-step AI agent**.

Decision Logic

Agent's Decision-Making Flowchart

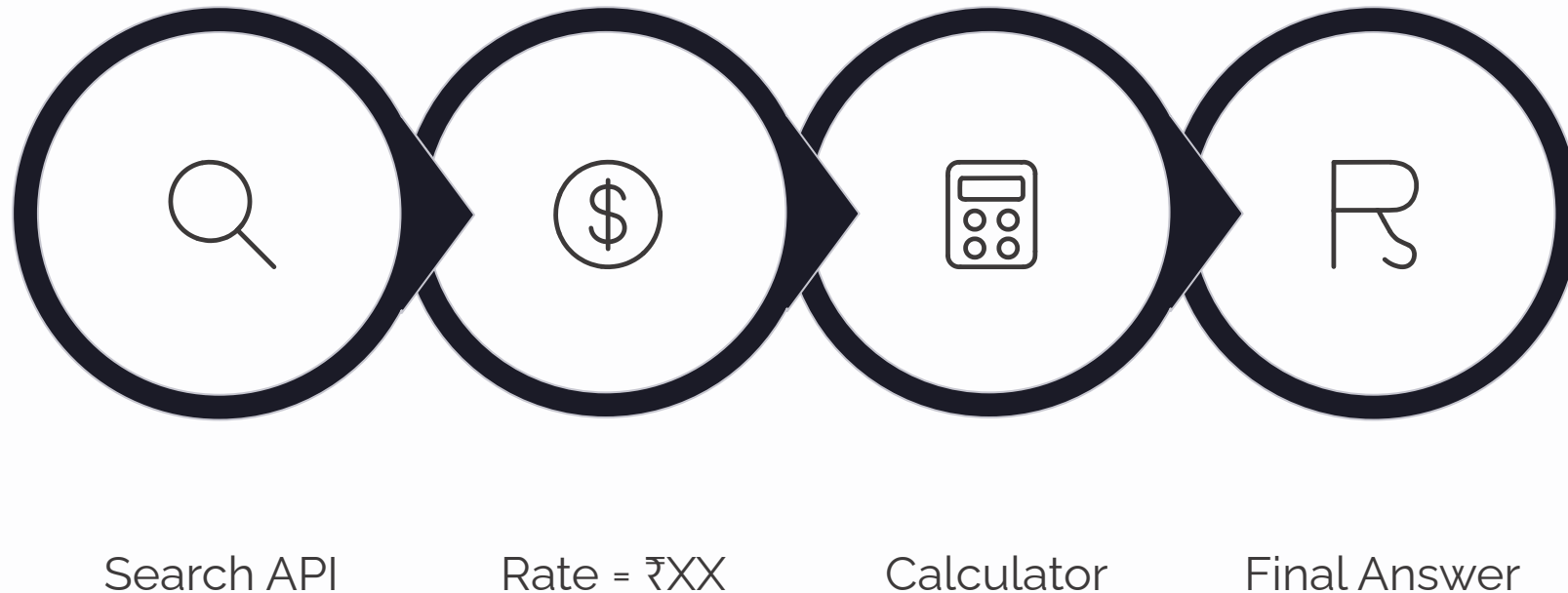
The agent loops: **Receive Question** → **Understand** → **Is a tool required?** If NO → generate answer directly. If YES → Select Tool → Execute → Get Result → Is another step needed? If YES → select next tool. If NO → Final Answer.

— Dr. Anilkumar V. Brahmane | Computer Engineering | Sanjivani College of Engineering



Search + Calculator: Advanced Example

User Question: "Find the current exchange rate of USD to INR and calculate the INR value of \$100."



When the **output of one tool becomes the input for another**, it is called **Tool Chaining** — a critical concept in Agentic AI. This demonstrates: Reasoning + Tool Selection + Execution + Intermediate Result + Final Answer.

Engineering Example

Multi-Step Problem: Motor Energy Cost

1

Calculate Energy

Power = 2 kW, Time = 6 hrs

Energy = $2 \times 6 = 12$ kWh

2

Calculate Cost

Rate = ₹8/kWh

Cost = $12 \times ₹8 = ₹96$

3

Final Answer

Daily energy = **12 kWh**

Daily cost = **₹96**

The agent uses the **Calculator tool for both steps**, using the intermediate result of Step 1 as input for Step 2. This is multi-step reasoning in action.

Implementation

Technology Stack & Project Structure

Recommended Stack

- **Language:** Python
- **LLM API:** Gemini / OpenAI / any LLM
- **Tools:** Calculator + Search
- **Interface:** CLI first, then Streamlit

Project Structure

```
tool_using_agent/  
├── app.py  
├── tools/  
│   ├── calculator.py  
│   └── search.py  
├── agent/  
│   └── agent.py  
├── requirements.txt  
└── README.md
```

Beginners can start with a single `app.py` file.

Step 1 – Define the Tools (calculator.py & search.py)

Before building the agent, we define the tools it can call. Show two Python code blocks:

Tool 1 – Calculator Tool (calculator.py):

```
# tools/calculator.py

def calculator(expression: str) -> str:
    """
    Evaluates a mathematical expression safely.
    Input: expression as string e.g. "2 * 6"
    Output: result as string
    """
    try:
        result = eval(expression) # Use eval for simple math
        return str(result)
    except Exception as e:
        return f"Calculator Error: {str(e)}"
```

Tool 2 – Search Tool (search.py):

```
# tools/search.py

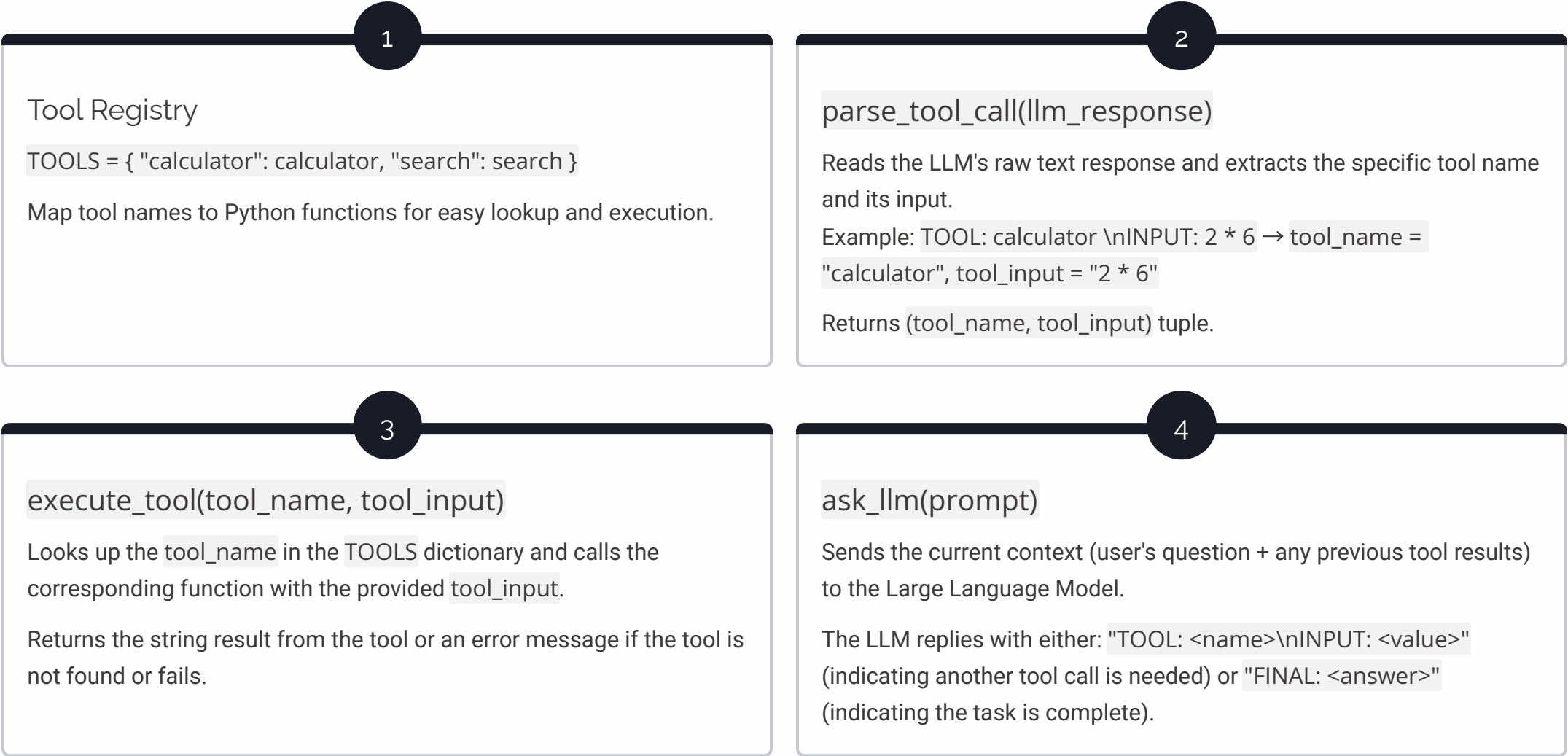
def search(query: str) -> str:
    """
    Simulates a web search. In real use, connect to
    Google Search API, SerpAPI, or DuckDuckGo API.
    Input: search query string
    Output: simulated or real search result string
    """

    # Simulated responses for demo purposes
    mock_data = {
        "USD to INR exchange rate": "1 USD = 83.5 INR",
        "electricity rate in India": "₹6 per kWh",
        "speed of light": "299,792,458 m/s",
    }
    for key in mock_data:
        if key.lower() in query.lower():
            return mock_data[key]
    return f"Search result for '{query}': No data found (use real API in production)"
```

In production, replace mock_data with a real API call using requests library.

The Brain: How the Agent Decides Which Tool to Use

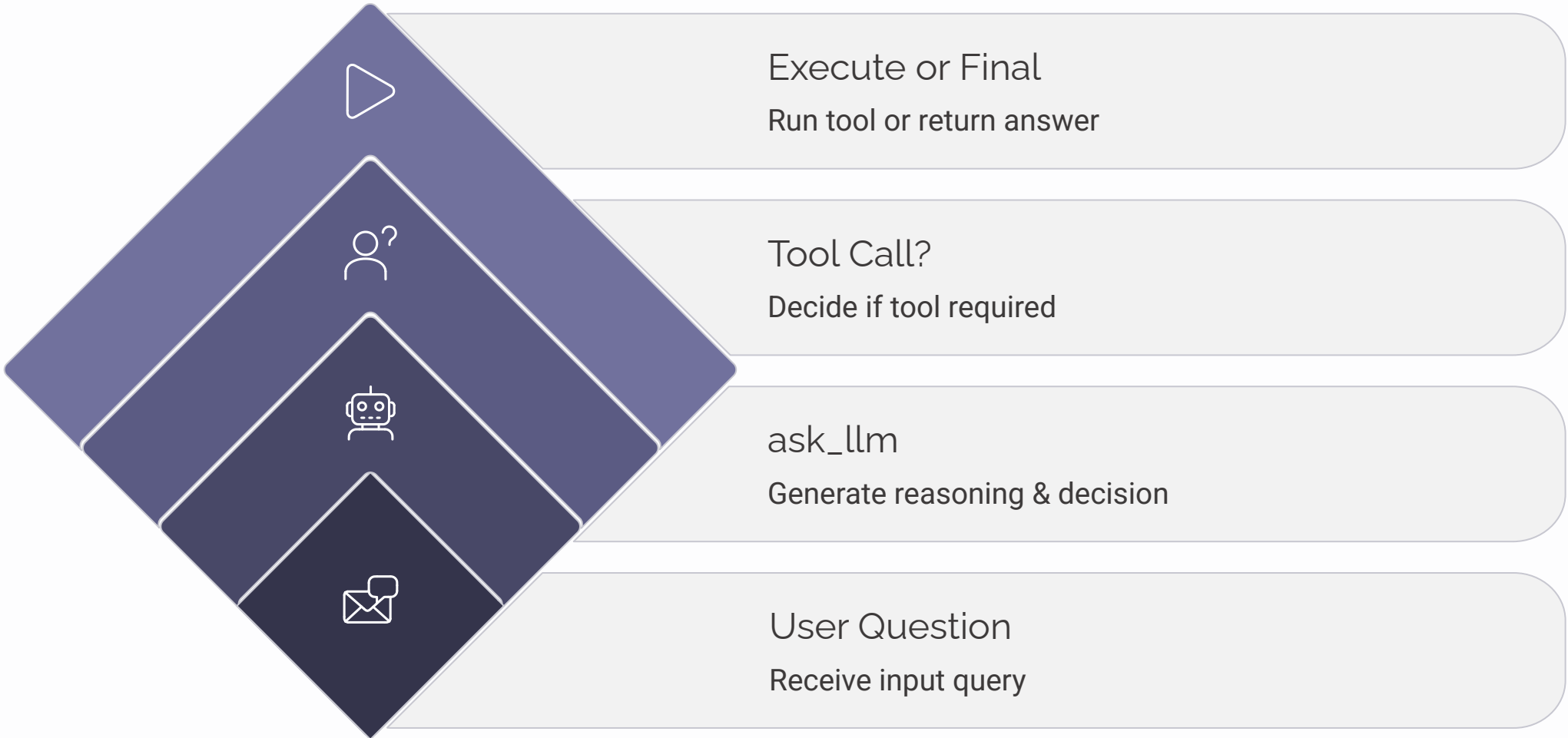
This `agent.py` file contains the core reasoning loop that enables the AI agent to understand requests, decide on appropriate actions, and execute them effectively.



`run_agent(user_question, max_steps=5)` [THE MAIN LOOP]

- Build context = "Question: ..."
- Loop up to `max_steps`:
 - a. Call `ask_llm(context)`
 - b. If response starts with "FINAL:" → return answer
 - c. Else parse tool call → execute tool → append result to context
- If loop ends without FINAL → return "Max steps reached"

This flowchart illustrates the agent's dynamic decision-making process.



Full Source Code with Line-by-Line Explanation

Section A — Imports and Tool Registry:

```
from tools.calculator import calculator
from tools.search import search

TOOLS = {
    "calculator": calculator,
    "search": search,
}
```

Import both tool functions. TOOLS dict maps string names to functions so the agent can call them by name.

Section B — parse_tool_call function:

```
def parse_tool_call(llm_response: str):
    lines = llm_response.strip().splitlines()
    tool_name = None
    tool_input = None
    for line in lines:
        if line.startswith("TOOL:"):
            tool_name = line.replace("TOOL:", "").strip()
        if line.startswith("INPUT:"):
            tool_input = line.replace("INPUT:", "").strip()
    return tool_name, tool_input
```

Splits LLM response line by line. Looks for lines starting with "TOOL:" and "INPUT:" and extracts their values.

Section C — execute_tool function:

```
def execute_tool(tool_name: str, tool_input: str) -> str:
    if tool_name in TOOLS:
        return TOOLS[tool_name](tool_input)
    return f"Unknown tool: {tool_name}"
```

Looks up the tool in the TOOLS dict and calls it. If tool not found, returns an error string.

Section D — ask_llm function (simulated for demo):

```
def ask_llm(prompt: str) -> str:
    prompt_lower = prompt.lower()
    if "exchange rate" in prompt_lower and "calculate" in prompt_lower:
        if "tool result" not in prompt_lower:
            return "TOOL: search\nINPUT: USD to INR exchange rate"
        else:
            return "TOOL: calculator\nINPUT: 100 * 83.5"
    if "energy" in prompt_lower or "kwh" in prompt_lower:
        if "tool result" not in prompt_lower:
            return "TOOL: calculator\nINPUT: 2 * 6"
        else:
            return "TOOL: search\nINPUT: electricity rate in India"
    return "FINAL: I don't have enough information."
```

In a real project, replace this with an actual API call to Gemini or OpenAI. This simulated version uses keyword matching to mimic LLM decision-making for classroom demo.

Section E — run_agent main loop:

```
def run_agent(user_question: str, max_steps: int = 5) -> str:
    context = f"Question: {user_question}"
    for step in range(1, max_steps + 1):
        llm_response = ask_llm(context)
        if llm_response.startswith("FINAL:"):
            return llm_response.replace("FINAL:", "").strip()
        tool_name, tool_input = parse_tool_call(llm_response)
        if not tool_name:
            return "Agent could not determine next action."
        tool_result = execute_tool(tool_name, tool_input)
        context += f"\nTool: {tool_name}\nInput: {tool_input}\nResult: {tool_result}"
    return "Max steps reached."
```

This is the ReAct loop. Each iteration: ask LLM → check if done → call tool → update context → repeat.

Step 4 – The Entry Point (app.py) & Live Execution Trace

Running the Agent and Reading the Output

Section 1 — app.py (the entry point):

```
from agent.agent import run_agent if __name__ == "__main__":  print("=== AI Tool-Using Agent ===\n")  # Test 1: Single-step — pure calculator
q1 = "Calculate 25 * 4"  answer1 = run_agent(q1)  print(f"Q: {q1}\nA: {answer1}\n")  # Test 2: Multi-step — search then calculate  q2 = "Find
the exchange rate of USD to INR and calculate the INR value of $100"  answer2 = run_agent(q2)  print(f"Q: {q2}\nA: {answer2}\n")  # Test 3:
Multi-step — calculate then search  q3 = "A motor runs at 2 kW for 6 hours. What is the energy cost in India?"  answer3 = run_agent(q3)
print(f"Q: {q3}\nA: {answer3}\n")
```

Explanation: app.py is the main entry point. It imports run_agent and tests it with 3 different questions.

Section 2 — Live Execution Trace for Test 2 (Multi-step):

Show as a step-by-step trace table with columns: Step | Action | Input | Output

Step	Action	Input	Output
Step 1	ask_llm	"Find exchange rate of USD to INR and calculate \$100"	TOOL: search, INPUT: USD to INR exchange rate
Step 2	execute_tool (search)	"USD to INR exchange rate"	"1 USD = 83.5 INR"
Step 3	ask_llm	context + tool result	TOOL: calculator, INPUT: 100 * 83.5
Step 4	execute_tool (calculator)	"100 * 83.5"	"8350.0"
Step 5	ask_llm	context + tool result	FINAL: \$100 = ₹8350

Section 3 — How to Run:

```
pip install openai (or google-generativeai for Gemini) python app.py
```

 For beginners: put all code in a single app.py file. No folder structure needed to get started.

Engineering Applications

Real-World Use Cases by Branch



Mechanical

Machine efficiency, power consumption, production rate, material requirements.



Civil

Material estimation, concrete quantity, construction cost, area/volume.



Electrical

Voltage/current calculations, power, energy consumption, electricity cost.



Computer / Mechatronics

API calls, database queries, sensor data analysis, motor calculations, AI assistants.

Test Your Agent

Recommended Test Cases

Test Case	Question	Expected Tool
1 – Calculator	Calculate 125×24	Calculator
2 – Percentage	18% of ₹5000	Calculator
3 – Search	What is Industry 4.0?	Search Tool
4 – Multi-Step	200 units/day + 15% increase → new production?	Calculator → Intermediate → Final
5 – Tool Chaining	Find component price → cost of 50 units	Search → Calculator → Total Cost

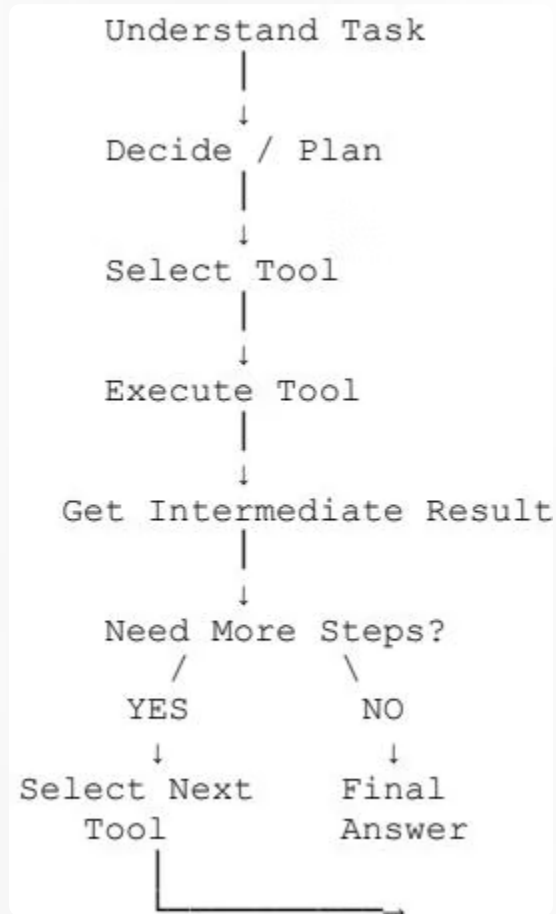
Marks Rubric & Submission Checklist

Evaluation Rubric (Total: 100)

Component	Marks
Understanding of AI Agent	10
Tool Design (Calculator + Search)	25
Tool Selection Logic	15
Multi-Step Problem Solving	15
Intermediate Results + Final Response	15
Testing, Documentation & Demo	20

What to Submit

- Source Code
- README / Documentation
- Architecture Diagram
- Tool Descriptions
- 4–5 Test Cases with Screenshots
- Short Video Demonstration



Key Learning Outcome

"An AI agent is not just an LLM that generates text. It can understand a task, decide what action is required, select an appropriate tool, execute that tool, use the returned information, perform additional actions when necessary, and finally generate a useful response."

Formula to Remember

AI Agent = LLM + Decision Making + External Tools + Tool Results + Multi-Step Solving + Final Response

Dr. Anilkumar V. Brahmane | Computer Engineering | Sanjivani College of Engineering